Final project

AE 6042 - Computational Fluid Dynamics

Baptiste Romain

Introduction

During the lectures, we studied different concepts used in computational fluid dynamics to ensure stability and accuracy of the solution. This final project aims at coding from scratch a solver of the Navier-Stokes equation on a structured mesh. Thus, we will have to deepen our understanding of the different concepts and adapt them to our particular requirements.

This report first sums up the main equations used to achieve the goal of each step of the project. Then, the results of the code written are exposed, compared to hand calculations and discussed to understand the impact of each concept used on the final result.

Table of contents

Introduction 2
1. Problem Formulation 4
Project Description 4
Governing equations 4
Numerical method 4
Step 1: Inviscid 1st order Euler simulation 6
Step 2-3: Higher order inviscid Euler simulation 7
Step 4-5: Higher order Navier-Stokes simulation 7
2. Results and discussion 9
Overview of work achieved 9
Step 1 9
Step 2-3 10
Conclusion 12
Appendix A: Interpolation file (MUSCL scheme) 13
Appendix B: set file (initial and boundary conditions) 15
Appendix C: extract file 17
Appendix D: plot file 20
Appendix E: save file 22
Appendix F: step_1_AUSM file (computes only one time step) 24
Appendix G: step_1 file (full computational loop) 29
Appendix H: initial and physical conditions 32
Appendix I: Computer architecture and language used 33

1. Problem Formulation

Project Description

The goal of this project is to develop a 2-D fluid dynamics solver working on structured meshes. It is divided in multiple steps aiming to stepping up complexity little by little. First by coding low order Euler equation solver and then by adding higher order accuracy and friction effects.

Governing equations

The inviscid Euler equation solved in the first parts of the problem is the following:

$$\frac{\partial \hat{Q}}{\partial t} = -\frac{\partial \hat{E}_\xi}{\partial \xi} - \frac{\partial \hat{F}_\eta}{\partial \eta}$$

Where $\hat{Q} = \Delta V \begin{bmatrix} \rho \\ \rho u \\ \rho v \\ \rho e_t \end{bmatrix}$, $\hat{E}_\xi = \begin{bmatrix} \rho u_\xi \\ \rho u u_\xi + n x_\xi P \\ \rho v u_\xi + n y_\xi P \\ \rho h_t u_\xi \end{bmatrix}$ and $\hat{F}_\eta = \begin{bmatrix} \rho v_\eta \\ \rho u v_\eta + n x_\eta P \\ \rho v v_\eta + n y_\eta P \\ \rho h_t v_\eta \end{bmatrix}$

We also define the contravariant velocities:

$$u_\xi = n x_\xi u + n y_\xi v$$

$$v_\eta = n x_\eta u + n y_\eta v$$

Numerical method

To numerically solve the equation, we use backward time stepping and a finite volumes formulation. Hence, the equation we are solving is the following:

$$\hat{Q}_{i,j}^{n+1} = \hat{Q}_{i,j}^{n} - \Delta\tau \left[\hat{E}_\xi S_\xi \Big|_{i-\frac{1}{2},j}^{i+\frac{1}{2},j} + \hat{F}_\eta S_\eta \Big|_{i,j-\frac{1}{2}}^{i,j+\frac{1}{2}} \right]$$

The following logic has been implemented to solve the problem:

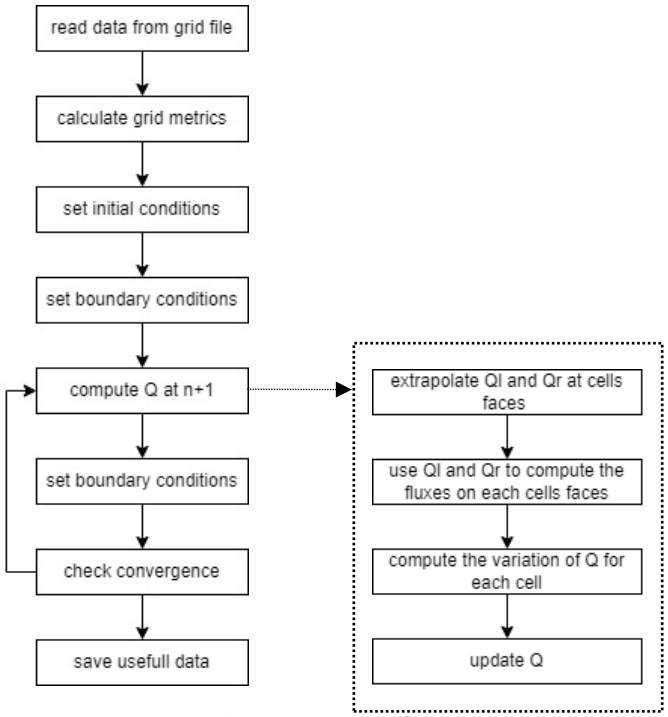


Figure 1: step by step procedure for computation

Initial Conditions:

The initial conditions are set uniformly on all the computational space and each Q variable is set equals to what they are at inlet condition, which can easily be calculated give the following data: P=101325Pa, M=2.0 along the x direction and T=300K and the formulas used to compute inlet conditions.

Boundary conditions:

- Upper wall

For This wall, it was stated in the project description that we had to implement a slip wall adiabatic condition. In the lectures notes, it was also stated that a 0 pressure gradient was often implemented on walls.

Thus, as it was a flat wall, the slip wall condition was simplified and was expressed as follows:

$$(\rho u)_{i,ny} = \rho_{i,ny} u_{i,ny-1}$$

$$(\rho v)_{i,ny} = -\rho_{i,ny} v_{i,ny-1}$$

For the adiabatic condition, the following formula was implemented and solved for e_t:

$$T_{i,ny} = T_{i,ny-1}$$

Where $T_{i,j} = T(Q_{i,j}) = \frac{(\gamma-1)}{R}\left(e_t - \frac{1}{2}(u^2+v^2)\right)_{i,j}$

Finally, the 0 pressure gradient was implemented and again solved for e_t:

$$P_{i,ny} = P_{i,ny-1}$$

Where $P_{i,j} = P(Q_{i,j}) = (\gamma-1)\left(\rho e_t - \frac{1}{2}\rho(u^2+v^2)\right)_{i,j}$

- Lower wall + airfoil

For this wall, for the first inviscid parts, the same no pressure gradient, adiabatic and slip wall conditions are applied.

The first two are the same as above but applied in j = 0 and j = 1 instead of j = ny and j = ny-1.

For the slip wall, as the wall is not flat, the condition is a bit different and is the following:

$$\begin{cases} u_{i,0} = u_{i,1}(n_y^2 - n_x^2) - 2n_x n_y v_{i,1} \\ v_{i,0} = v_{i,1}(n_x^2 - n_y^2) - 2n_x n_y u_{i,1} \end{cases}$$

Where n_x and n_y are the x and y component of the vector tangent to the wall.

- Inlet

At the inlet, the flow properties are given and constant over time. However, they have to be converted into the state variables used in Q. Hence, as we know that P=101325Pa, M=2.0 along the x direction and T=300K, we can derive:

$$Q_{inlet} = \begin{bmatrix} \frac{P}{RT} \\ M\sqrt{\gamma RT} \\ 0 \\ \frac{P}{\gamma - 1} + \frac{1}{2}\rho M^2 \gamma RT \end{bmatrix}$$

As those conditions are constant over time, the inlet hallo cells do not have to be updated at each iteration.

- Outlet

For the outlet, after each iteration, the last column of hallo cells is set equal to the column just before.

Step 1: Inviscid 1st order Euler simulation

The heart of this project is the method used to update Q. In the first part, we had to implement a first order accurate method. To do so, it is mandatory to use a flux reconstruction method that includes numerical dissipation for the solution to be stable.

The method chosen here is the AUSM scheme.

First order interpolation:

As the method is first order, the simple following formula is used:

$$\begin{cases} Q^l_{i+\frac{1}{2},j} = Q_{i,j} \\ Q^r_{i+\frac{1}{2},j} = Q_{i+1,j} \end{cases}$$

And the same idea is used in the j direction.

Flux reconstruction:

The general formula will be given for the flux along i. To have the flux along j, the only modification to do is to change n_x and n_y.

First, the flux is decomposed in two parts:

$$E_\xi = \begin{pmatrix} \rho u_\xi \\ \rho u u_\xi \\ \rho v u_\xi \\ \rho h_t u_\xi \end{pmatrix} + \begin{pmatrix} 0 \\ n_x P \\ n_y P \\ 0 \end{pmatrix}$$

Where $u_\xi = n_x u + n_y v$

Then, we compute the Mach number based on u_ξ: $M_{L/R} = \frac{u_\xi^{L/R}}{c^{L/R}}$ with c being the speed of sound and L/R describe either left of right state. We take the plus or minus value as follows:

$$\begin{cases} M_L^+ = M^+(M_L) \\ M_R^- = M^-(M_R) \end{cases}$$

Where $M^\pm = \begin{cases} \pm\frac{1}{4}(M \pm 1)^2 \; if \; |M| \leq 1 \\ \frac{1}{2}(M \pm |M|) \; if \; |M| > 1 \end{cases}$

Finally, the first term of the flux can be computed like this:

$$\begin{pmatrix} \rho u_\xi \\ \rho u u_\xi \\ \rho v u_\xi \\ \rho h_t u_\xi \end{pmatrix} = \frac{1}{2} M_{1/2} \left[\begin{pmatrix} \rho c \\ \rho u c \\ \rho v c \\ \rho h_t c \end{pmatrix}_L + \begin{pmatrix} \rho c \\ \rho u c \\ \rho v c \\ \rho h_t c \end{pmatrix}_R \right] - \frac{1}{2} |M_{1/2}| \left[\begin{pmatrix} \rho c \\ \rho u c \\ \rho v c \\ \rho h_t c \end{pmatrix}_r - \begin{pmatrix} \rho c \\ \rho u c \\ \rho v c \\ \rho h_t c \end{pmatrix}_L \right]$$

With $M_{1/2} = M_L^+ + M_R^-$

The pressure term is computed with a similar idea:

$$\begin{pmatrix} 0 \\ n_x P \\ n_y P \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ n_x \left(P^+(P_L) + P^-(P_R) \right) \\ n_y \left(P^+(P_L) + P^-(P_R) \right) \\ 0 \end{pmatrix}$$

With $P^\pm = \begin{cases} \pm \frac{P}{2} \left(1 \pm M_{1/2} \right) \; if \; |M| \leq 1 \\ \frac{P}{2} \frac{\left(M_{1/2} \pm |M_{1/2}| \right)}{M_{1/2}} \quad if \; |M| > 1 \end{cases}$

In the end, we can update the cell's value with:

$$\hat{Q}_{i,j}^{n+1} = \hat{Q}_{i,j}^n - \Delta\tau \left[\hat{E}_\xi S_\xi \Big|_{i-\frac{1}{2},j}^{i+\frac{1}{2},j} + \hat{F}_\eta S_\eta \Big|_{i,j-\frac{1}{2}}^{i,j+\frac{1}{2}} \right]$$

Step 2-3: Higher order inviscid Euler simulation

For those cases, the formula used to compute the fluxes is the definition given in the governing equations with no modification. The part that must be changed is the interpolation scheme. In this code, the MUSCL scheme was implemented. At each cell face, the left and right state are computed as follows:

$$\begin{cases} Q^l_{i+\frac{1}{2},j} = Q_{i,j} + \frac{\epsilon}{4} \left[(1-\kappa)(Q_{i,j} - Q_{i-1,j}) \phi \left(\frac{fw}{bw} \right) + (1+\kappa)(Q_{i+1,j} - Q_{i,j}) \phi \left(\frac{bw}{fw} \right) \right] \\ Q^r_{i+\frac{1}{2},j} = Q_{i,j} - \frac{\epsilon}{4} \left[(1+\kappa)(Q_{i+1,j} - Q_{i,j}) \phi \left(\frac{fw}{bw} \right) + (1-\kappa)(Q_{i+2,j} - Q_{i+1,j}) \phi \left(\frac{bw}{fw} \right) \right] \end{cases}$$

For the boundaries of the computational domain, the left and right states are computed with a 1st order approximation. Also, we can set $\epsilon = 1$ and $\kappa = -1$.

For step 2, ϕ is set equals to 1.

For step 3, phi is the minmod limiter. We define $fw = \begin{cases} Q_{i+1,j} - Q_{i,j} & if \; (Q_{i+1,j} - Q_{i,j}) \neq 0 \\ 10^{-16} & otherwise \end{cases}$ and $bw = \begin{cases} Q_{i,j} - Q_{i-1,j} & if \; (Q_{i,j} - Q_{i-1,j}) \neq 0 \\ 10^{-16} & otherwise \end{cases}$. Then, if only one of bw or fw is equals to 0, the result is really close to what it should be, and if both are 0, the result is 1.

The minmod limiter itself is defined by:

$$minmod(r) = \max(0, \min(1, r))$$

Step 4-5: Higher order Navier-Stokes simulation

To implement step 4, we have to add the viscous term in the evolution equation that becomes:

$$\hat{Q}_{i,j}^{n+1} = \hat{Q}_{i,j}^{n} - \Delta\tau\left[\left(\hat{E}_{\xi} - \hat{E}_{v\xi}\right)S_{\xi}\Big|_{i-\frac{1}{2},j}^{i+\frac{1}{2},j} + \left(\hat{F}_{\eta} - \hat{F}_{v\eta}\right)S_{\eta}\Big|_{i,j-\frac{1}{2}}^{i,j+\frac{1}{2}}\right]$$

Where the new viscous components are functions of $Q_v = \begin{pmatrix}\rho\\ u\\ v\\ T\end{pmatrix}$:

$$\hat{E}_{v\xi} = R_{\xi\xi}\frac{\partial Q_v}{\partial \xi} + R_{\xi\eta}\frac{\partial Q_v}{\partial \eta}$$

$$\hat{F}_{v\eta} = R_{\eta\xi}\frac{\partial Q_v}{\partial \xi} + R_{\eta\eta}\frac{\partial Q_v}{\partial \eta}$$

With $R_{mn} = \frac{1}{\Delta V}\begin{bmatrix}0 & 0 & 0 & 0\\ 0 & r_{22} & r_{23} & 0\\ 0 & r_{32} & r_{33} & 0\\ 0 & r_{42} & r_{43} & r_{44}\end{bmatrix}, r_{22} = \frac{\mu}{S_m}\left(\frac{4}{3}S_{m_x}S_{n_x} + S_{m_y}S_{n_y}\right), r_{23} = \frac{\mu}{S_n}\left(-\frac{2}{3}S_{m_x}S_{n_y} + S_{m_y}S_{n_x}\right), r_{32} = \frac{\mu}{S_m}\left(-\frac{2}{3}S_{m_y}S_{n_x} + S_{m_x}S_{n_y}\right), r_{33} = \frac{\mu}{S_m}\left(\frac{4}{3}S_{m_y}S_{n_y} + S_{m_x}S_{n_x}\right), r_{42} = \frac{\mu}{S_m}\left[u\left(\frac{4}{3}S_{m_x}S_{n_x} + S_{m_y}S_{n_y}\right) + v\left(-\frac{2}{3}S_{m_y}S_{n_x} + S_{m_x}S_{n_y}\right)\right], r_{43} = \frac{\mu}{S_m}\left[u\left(-\frac{2}{3}S_{m_x}S_{n_y} + S_{m_y}S_{n_x}\right) + v\left(\frac{4}{3}S_{m_y}S_{n_y} + S_{m_x}S_{n_x}\right)\right], \frac{\lambda}{S_m}\left(S_{m_y}S_{n_x} + S_{m_y}S_{n_y}\right)$

The viscosity and thermal diffusivity are computed with the Suterland's formulas.

For boundary conditions, we need to modify the cells representing the airfoil to have 0 velocity at the wall. Furthermore, for the last case, we have to use the thermal diffusivity to apply an isothermal condition over the airfoil.

2. Results and discussion

Overview of work achieved

For this project, only the first case is converged although it seems that the result still has some mistakes in it. For the subsequent cases, I was thus only able to work on cases 2 and 3, but not on the full Navier-Stokes equations.

Step 1

The simulation using the grid g65x49u gives the following results (let's note that the boundary cells are also plotted):

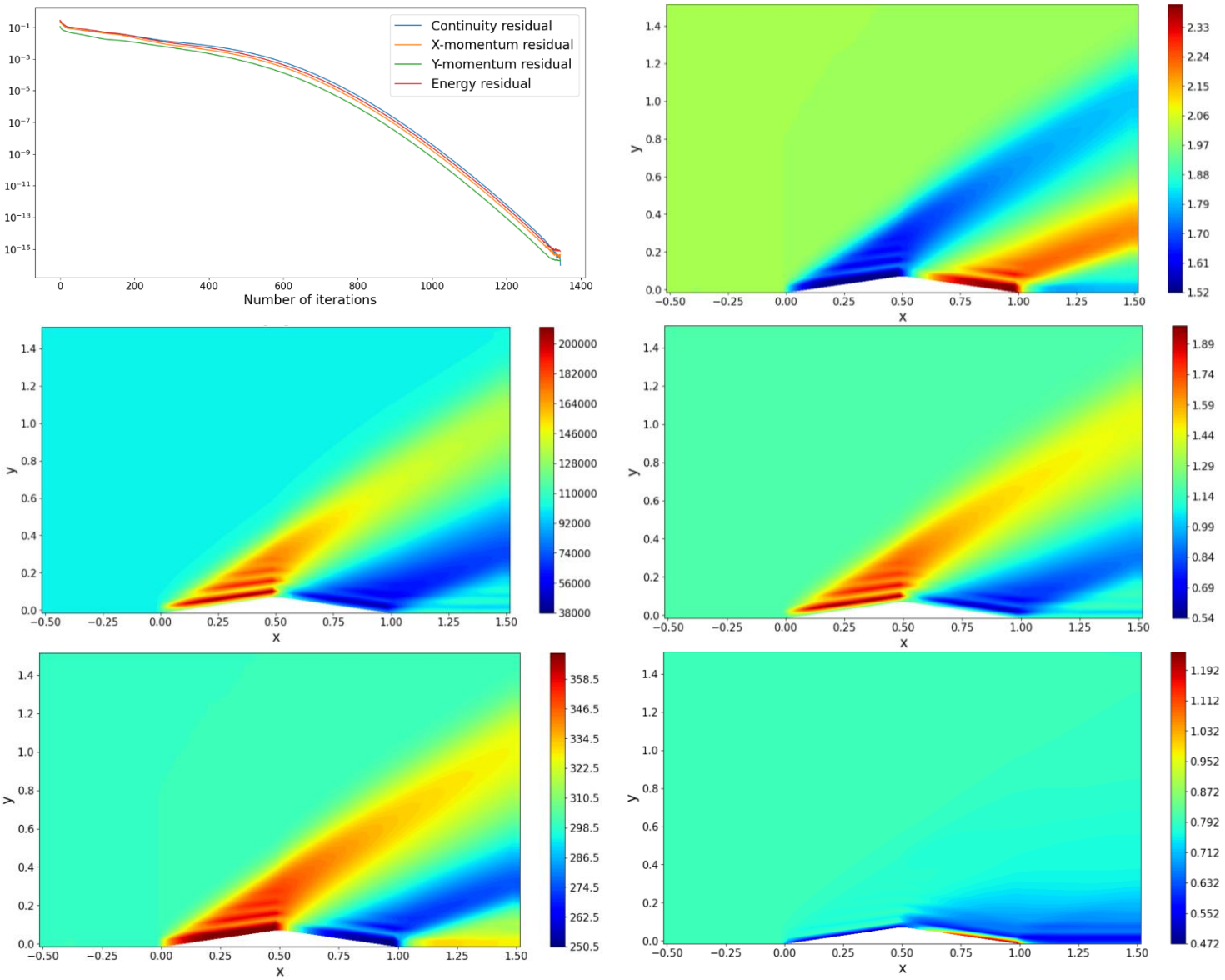


Figure 2: plots of the step 1 simulation (AUSM scheme) plotted with boundary cells. From top to bottom and from left to right: residuals, Mach number, pressure, density, temperature and stagnation pressure

First, we can note that the residual (L_2) plot is monotonicaly decreasing wich means that the schem is TVD. However, we can also see somebig non physical oscilations along the plot. At first, I thought those oscilations originated in the numerical discipation used to make the first order scheme work. But after comparing those results with some others, I figured out that it was actually due to some mistake in the codes that I weren't able to find. Nontheless, I tried to run the solver on a carthesian mesh and it

holded the free flow, so the error does not come from the solver itself. Moreover, after multiple tries and after analysing non converginig results, I think that the mistake comes from a part I coded before the heart of the solver, like boundary conditions. But we will talk more deeply about that in a coming part.

So to make the following table, the values used are values of inside the shock, just after the non physical oscilations ended.

	Wave angle	M_2	P2/P1	rho2/rho1	T2/T1	P02/P01
Hand calculations	39.31	1.641	1.707	1.458	1.170	0.9846
Computer solutions	37.50	1.637	1.710	1.459	1.172	0.9814
Error relative to the hand calculation	4.59%	0.244%	0.176%	0.069%	0.171%	0.325%

We can thus see that despite the oscilations, the results given by the simulation are really close and consistent with the hand calculations. Actually in this case (adiabatic and inviscide, the relative error should tend to be 0, and that really is the case here.

One other result that is interesting to consider is the following:

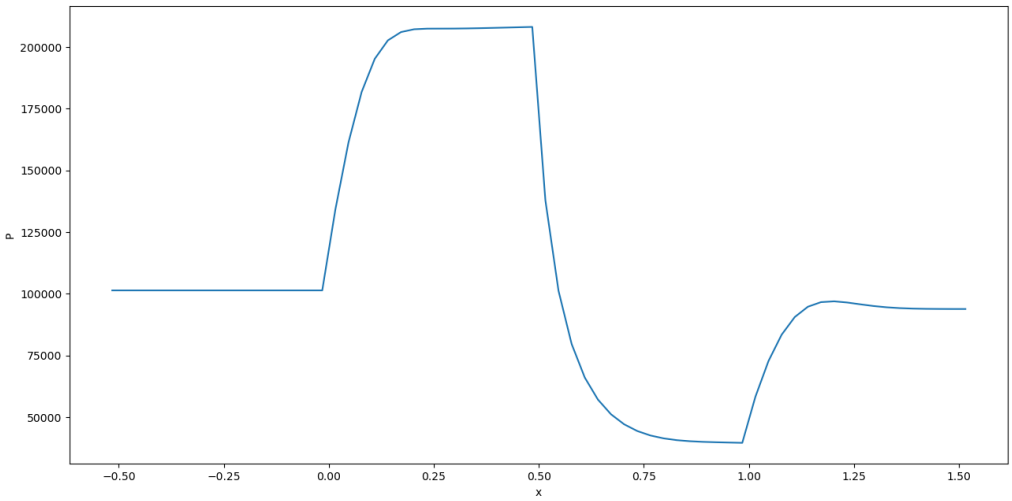


Figure 3: Pressure distribution along the bottom wall

We first have to note that the theoretical pressure profile allong the bottom wall should be a step fuction. Here we can see the profile resulting form the first order simulation. As stated before, this scheme relies on numerical discipation. This results in the pressure profile being smoother and also on overshooting in the last section. Althought I do not have other cases to compare it to, one the the goal of the simulation would be to have this graph looking as much as possible to a step graph.

Step 2-3

For those two cases, there are no plots as the simulation did not converge. Actually, if we look at the evolution of the computational domain over the first few iterations, we can see that the problem

comes from oscillations propagating upwards and originating from the downstream part of the airfoil. As this behavior is exactly what also happens with the first case, I think that the script used to compute the fluxes for this part is actually correct, but that there is a function shared by both codes that is wrong.

However, we can expect the contour plots to look very similar (without the oscillations) and the results compared to the hand computed ones to be a bit more accurate.

For the plot of pressure along the bottom wall, I think that without the flux limiters, we will see a sharper plot, closer to a step function, but with a lot of overshoots near the high gradient zone, leading to oscillations next to these zones. This would be corrected by the flux limiter that would highly reduce or remove those nonphysical oscillations but still allow for high gradients.

To conclude this part, we can expect that increasing the accuracy of the scheme will lead to results closer and closer to the hand calculations that are done considering inviscide flow with adiabatic walls. Using flux limiters will also help to avoid having numerical discipation or oscilations.

When adding the viscous terms, we can expect the results to diverge a bit form the hand calculations as the hypothesis made won't hold anymore. Firstly the inviscide flow asumption is removed, and in the second case, the adiabatic asumption is also removed. Finaly, we should see a boundary layer appearing on the surface of the wing.

Conclusion

To conclude on the realization of the overall project, the biggest issue is the presence of those oscillations propagating upwards. Throughout my tests, I found that when not considering the F flux (quasi 1D problem), the result always converges, but when adding the F flux, it diverges. Thus, I should look more deeply into mistakes in this part. When found, the code will surely work.

Also, the flux limiter can be implemented using the value of pressure for instance instead of the raw Q values to increase stability. This can be a way of improving the code.

Finally, when I started this project, I was lost with all the vocabulary used and the different steps and formulas that had to be used to do this project. I think my mistakes comes from this fact. I think that I misunderstood some formulas (certainly one involving nx and ny that understood badly at first) that are implemented wrongly and that are now hidden in the code and so really hard to debug. Now, my understanding of the subject is incredibly better and I think that if I were to do the project from scratch again, I would implement the good formulas and know how to test them independently to finally have something work fine.

Appendix A: Interpolation file (MUSCL scheme)

```
########################################################################
########################  flux limiters  ###############################
########################################################################
import numpy as np

def minmod(r):
    min= 1*(r>=1) + (r<1)*r
    max = 0*(min<=0) + (min>0)*min
    return(max)

def van_albada_1_flux_limiter(r):
    return((r**2+r)/(r**2+1))

def van_albada_2_flux_limiter(r):
    return(2*r/(r**2+1))

def lax_wendroff(r):
    return 1

def beam_warming(r):
    return r


def MUSCL(X,eps=0,k=-1,phi=lax_wendroff):
    """
    Return the interpolation of cells state on cell faces in the
computational domain (boundary included)
    :param X: matrix storing the state vectors (3D-array)
    :param eps: epsilon parameter of the MUSCL scheme as described in the
lecture notes
    :param k: k parameter of the MUSCL scheme as described in the lecture
notes
    :return: interpolated right and left state vectors (3D-arrays)
    """
    # for the moment, only first order is working
    # reading of the neighbor cells #
    bw_x = X[1:-1,1:-1,:]-X[1:-1,:-2,:]
    fw_x = X[1:-1,2:,:]-X[1:-1,1:-1,:]
    bw_x = (bw_x!=0)*bw_x + (bw_x==0)*1e-16
    fw_x = (fw_x!=0)*fw_x + (fw_x==0)*1e-16
    Q_l_x_plus_half = X[1:-1,1:-1,:] + eps/4*((1-k)*phi(fw_x/bw_x)*(X[1:-
1,1:-1,:]-X[1:-1,:-2,:])+(1+k)*phi(bw_x/fw_x)*(X[1:-1,2:,:]-X[1:-1,1:-
1,:]))  # forgetting the leftmost cells
    Q_l_x_plus_half = np.hstack(((X[1:-
1,0,:])[:,np.newaxis,:],Q_l_x_plus_half))
    Q_r_x_plus_half = X[1:-1,1:-1,:] - eps/4*((1+k)*phi(fw_x/bw_x)*(X[1:-
1,1:-1,:]-X[1:-1,:-2,:])+(1-k)*phi(bw_x/fw_x)*(X[1:-1,2:,:]-X[1:-1,1:-
1,:]))  # forgetting the rightmost cells
    Q_r_x_plus_half = np.hstack((Q_r_x_plus_half,(X[1:-1,-
1,:])[:,np.newaxis,:]))

    bw_y = X[1:-1,1:-1,:]-X[:-2,1:-1,:]
    fw_y = X[2:,1:-1,:]-X[1:-1,1:-1,:]
    bw_y = (bw_y != 0) * bw_y + (bw_y == 0) * 1e-16
    fw_y = (fw_y != 0) * fw_y + (fw_y == 0) * 1e-16
    Q_l_y_plus_half = X[1:-1,1:-1,:] + eps/4*((1-k)*phi(fw_y/bw_y)*(X[1:-
1,1:-1,:]-X[:-2,1:-1,:])+(1+k)*phi(bw_y/fw_y)*(X[2:,1:-1,:]-X[1:-1,1:-
1,:]))  # forgetting the bottom cells
    Q_l_y_plus_half = np.vstack(((X[0,1:-
```

```
1,:])[np.newaxis,:,:],Q_l_y_plus_half))
    Q_r_y_plus_half = X[1:-1,1:-1,:] - eps/4*((1+k)*phi(fw_y/bw_y)*(X[1:-
1,1:-1,:]-X[:-2,1:-1,:])+(1+k)*phi(bw_y/fw_y)*(X[2:,1:-1,:]-X[1:-1,1:-
1,:]))  # forgetting the top cells
    Q_r_y_plus_half = np.vstack((Q_r_y_plus_half,(X[-1,1:-
1,:])[np.newaxis,:,:]))


    return(Q_l_x_plus_half,Q_r_x_plus_half,Q_l_y_plus_half,Q_r_y_plus_half)
```

Appendix B: set file (initial and boundary conditions)

```
import numpy as np
from initial_conditions import *
import extract

def initial_conditions(nx,ny,bottom_surfaces):
    """
    sets all the interior points at the initial condition given in the
project assignment
    :param nx, ny: (int) size of the augmented grid which is not the number
of cells
    :param bottom_surfaces: (3D array) matrix containing the top and bottom
surface vectors
    :return:(3D array) Q
    """
    Q = np.zeros((ny-1,nx-1,4))

    Q[:, :, 0] += rhoini
    Q[:, :, 1] += rhoini*Mini*extract.c(gamma,Tini)
    Q[:, :, 2] += 0
    Q[:, :, 3] += Pini/(gamma-1) + rhoini*(Mini*extract.c(gamma,Tini))**2/2

    return(Q)

def bc(X,nx_eta,ny_eta):
    # inlet conditions
    X[:, 0, 0] = rhoini
    X[:, 0, 1] = rhoini * Mini * extract.c(gamma, Tini)
    X[:, 0, 2] = 0
    X[:, 0, 3] = Pini / (gamma - 1) + rhoini * (Mini * extract.c(gamma,
Tini)) ** 2 / 2



    X[:, -1, :] = X[:, -2, :]  # setting all the hallo cells on the right
to the value of the last column of computed cells

    # slip wall condition #
    # top wall
    #this approximation is fine as the top wall is flat, but in a second
time it could be verified more rigorously
    X[-1, 1:-1, 1] = X[-1, 1:-1, 0] * X[-2, 1:-1,1] / X[-2, 1:-1,0]  # slip
wall (the boudary condition is actualy computed only over the computational
domaine)
    X[-1, 1:-1, 2] = - X[-1, 1:-1, 0] * X[-2, 1:-1,2] / X[-2, 1:-1,0] #
slip wall (the boudary condition is actualy computed only over the
computational domaine)

    # bottom wall
    X[0,1:-1,1] = X[0, 1:-1, 0]*(extract.u(X)[1,1:-1]*(ny_eta[1,:]**2-
nx_eta[1,:]**2)-2*nx_eta[1,:]*ny_eta[1,:]*extract.v(X)[1,1:-1])
    X[0, 1:-1, 2] = X[0, 1:-1, 0] * (extract.v(X)[1, 1:-1] * (nx_eta[1, :]
** 2 - ny_eta[1, :] ** 2) - 2 * nx_eta[1, :] * ny_eta[1,:] *
extract.u(X)[1, 1:-1])

    # adiabatic condition #
    # top wall
    X[-1, :, 3] = r * X[-1, :, 0] / (gamma - 1) * ((gamma - 1) * (X[-2, :,
3] - 1 / 2 * (X[-2, :, 1] ** 2 + X[-2, :, 2] ** 2) / X[-2, :, 0]) / r / X[-
2, :, 0]) + 1 / 2 * (X[-1, :, 1] ** 2 + X[-1, :, 2] ** 2) / X[-1, :, 0]
```

```
    # bottom wall
    X[0, :, 3] = r * X[0, :, 0] / (gamma - 1) * ((gamma - 1) * (X[1, :, 3]
- 1 / 2 * (X[1, :, 1] ** 2 + X[1, :, 2] ** 2) / X[1, :, 0]) / r / X[1, :,
0]) + 1 / 2 * (X[0, :, 1] ** 2 + X[0, :, 2] ** 2) / X[0, :, 0]

    # 0 pressure gradient at walls #
    # bottom wall
    X[0, -1:1, 3] = (X[1, -1:1, 3] - 0.5 * (X[1, -1:1, 1] ** 2 + X[1, -1:1,
2] ** 2) / X[1, -1:1, 0]) + 0.5 * (X[0, -1:1, 1] ** 2 + X[0, -1:1, 2] ** 2)
/ X[0, -1:1, 0]
    # top wall
    X[-1, -1:1, 3] = (X[-2, -1:1, 3] - 0.5 * (X[-2, -1:1, 1] ** 2 + X[-2, -
1:1, 2] ** 2) / X[-2, -1:1, 0]) + 0.5 * (X[-1, -1:1, 1] ** 2 + X[-1, -1:1,
2] ** 2) / X[-1, -1:1, 0]

    return(X)
```

Appendix C: extract file

```
import numpy as np
import matplotlib.pyplot as plt
import cellproperties
import plotgrid
import readgrid
from air import *

#########################################################################
####################  physical data extraction  #########################
#########################################################################

def c(g,T):
    """
    Returns the value of speed of sound
    :param g: gamma at the points considered (float or 2D-array)
    :param T: temperature at a given point (float or 2D-array)
    :return: speed of sound (float or 2D-array)
    """
    return(np.sqrt(g*r*T))

def u(Q):
    """
    Extract the value of u form Q
    :param Q: array of state vectors (3D-array)
    :return: array containing the value of u (2D-array)
    """
    return(Q[:,:,1]/Q[:,:,0])

def v(Q):
    """
    Extract the value of v form Q
    :param Q: array of state vectors (3D-array)
    :return: array containing the value of v (2D-array)
    """
    return(Q[:,:,2]/Q[:,:,0])

def et(Q):
    """
    Extract the value of et form Q
    :param Q: array of state vectors (3D-array)
    :return: array containing the value of et (2D-array)
    """
    return (Q[:, :, 3] / Q[:, :, 0])

def P(Q):
    """
    Extract the value of P form Q
    :param Q: array of state vectors (3D-array)
    :return: array containing the value of P (2D-array)
    """
    Et = et(Q)
    U = u(Q)
    V = v(Q)
    return((gamma-1)*Q[:,:,0]*(Et-0.5*(U**2+V**2)))

def ht(Q):
    """
    Extract the value of ht form Q
    :param Q: array of state vectors (3D-array)
```

```
    :return: array containing the value of ht (2D-array)
    """
    Et = et(Q)
    p = P(Q)
    return(Et+p/Q[:, :, 0])

def T(Q):
    """
    Extract the value of T form Q
    :param Q: array of state vectors (3D-array)
    :return: array containing the value of T (2D-array)
    """
    return(((gamma - 1) * (Q[:, :, 3] - 1 / 2 * (Q[:, :, 1] ** 2 + Q[:, :,
2] ** 2) / Q[:, :, 0])) / r / Q[:, :, 0])

def M(Q):
    U = u(Q)
    V = v(Q)
    a = c(gamma,T(Q))
    return np.sqrt(U**2+V**2)/a

def P0(Q):
    p = P(Q)
    m = M(Q)
    return (p*(1+(gamma-1)/2*m**2)**(gamma/(gamma-1)))

######################################################################
##################  geometric data extraction  ######################
######################################################################

def direction_vectors(Sb,Sl):
    """
    Extrapolate the value of xi and eta in the middle of the cell
    :param Sb: Surface vectors of surfaces parallel to xi of the augmented
grid (3D-array)
    :param Sl: Surface vectors of surfaces parallel to eta of the augmented
grid (3D-array)
    :return: interpolated direction vectors components (2D-array)
    """
    # interpolation of surface vectors on centers of surfaces along i #
    Sb_mean = (Sb[:-1,:,:]+Sb[1:,:,:])/2
    Sb_mean_norm = np.linalg.norm(Sb_mean,axis=2)
    nx_xi = -Sb_mean[:,:,1]/Sb_mean_norm
    ny_xi = Sb_mean[:,:,0]/Sb_mean_norm

    Sl_mean = (Sl[:,0:-1,:]+Sl[:,1:,:])/2
    Sl_norm = np.linalg.norm(Sl_mean,axis = 2)
    nx_eta = Sl_mean[:,:,1]/Sl_norm
    ny_eta = -Sl_mean[:,:,0]/Sl_norm
    return(nx_xi,ny_xi,nx_eta,ny_eta)

def direction_vectors_i(Sb,Sl):
    """
    Extrapolate the value of xi and eta in the middle of vertical surfaces
    :param Sb: Surface vectors of surfaces parallel to xi of the augmented
grid (3D-array)
    :param Sl: Surface vectors of surfaces parallel to eta of the augmented
grid (3D-array)
    :return: interpolated direction vectors components (2D-array)
    """
    # interpolation of surface vectors on centers of surfaces along i #
```

```
    Sl_norm = np.linalg.norm(Sl[1:-1,1:-1,:],axis = 2)
    nx_xi = -Sl[1:-1,1:-1,0] / Sl_norm
    ny_xi = -Sl[1:-1,1:-1,1] / Sl_norm

    return(nx_xi,ny_xi)

def direction_vectors_j(Sb,Sl):
    """
    Extrapolate the value of xi and eta in the middle of horizontal
surfaces
    :param Sb: Surface vectors of surfaces parallel to xi of the augmented
grid (3D-array)
    :param Sl: Surface vectors of surfaces parallel to eta of the augmented
grid (3D-array)
    :return: interpolated direction vectors components (2D-array)
    """
    # interpolation of surface vectors on centers of surfaces along j #
    Sb_norm = np.linalg.norm(Sb[1:-1,1:-1,:],axis = 2)
    nx_eta = -Sb[1:-1,1:-1,0] / Sb_norm
    ny_eta = -Sb[1:-1, 1:-1, 1] / Sb_norm

    return (nx_eta, ny_eta)

########################################################################
######################  residuals extraction  ##########################
########################################################################

def residuals_L2(X,X0,Xini):
    """
    Computes the residuals in norm 2
    :param X: actual state vector (3D-array)
    :param X0: previous state vector (3D-array)
    :param Xini: initial state vector used for normalization (3D-array)
    :return: the value of residuals for each component (float)
    """
    rho_v=np.sqrt(Xini[:,:,1]**2+Xini[:,:,2]**2)
    cr = np.sqrt(np.sum(((X[:, :, 0] - X0[:, :, 0])/Xini[:,:,0]) ** 2))
    xr = np.sqrt(np.sum(((X[:, :, 1] - X0[:, :, 1])/rho_v) ** 2))
    yr = np.sqrt(np.sum(((X[:, :, 2] - X0[:, :, 2])/rho_v) ** 2))
    er = np.sqrt(np.sum(((X[:, :, 3] - X0[:, :, 3])/Xini[:,:,3]) ** 2))
    return(cr, xr, yr, er)

def residuals_Linf(X,X0,Xini):
    """
    Computes the residuals in norm inf
    :param X: actual state vector (3D-array)
    :param X0: previous state vector (3D-array)
    :param Xini: initial state vector used for normalization (3D-array)
    :return: the value of residuals for each component (float)
    """
    rho_v = np.sqrt(Xini[:, :, 1] ** 2 + Xini[:, :, 2] ** 2)
    cr = np.max((X[:, :, 0] - X0[:, :, 0])/Xini[:,:,0])
    xr = np.max((X[:, :, 1] - X0[:, :, 1])/rho_v)
    yr = np.max((X[:, :, 2] - X0[:, :, 2])/rho_v)
    er = np.max((X[:, :, 3] - X0[:, :, 3])/Xini[:,:,3])
    return(cr, xr, yr, er)
```

Appendix D: plot file

```
import matplotlib.pyplot as plt
import extract
from air import *

def residuals(Cr,Xr,Yr,Er):
    """
    Plot the residuals all together
    :param Cr: continuity residual (list)
    :param Xr: X-momentum residual (list)
    :param Yr: Y-momentum residual (list)
    :param Er: energy residual (list)
    """
    parameters = {'axes.labelsize': 20,
                  'axes.titlesize': 20,
                  'legend.fontsize': 20,
                  'xtick.labelsize': 15,
                  'ytick.labelsize': 15}
    plt.rcParams.update(parameters)
    plt.plot(Cr,label='Continuity residual')
    plt.plot(Xr, label='X-momentum residual')
    plt.plot(Yr, label='Y-momentum residual')
    plt.plot(Er, label='Energy residual')
    plt.legend()
    plt.semilogy()
    plt.xlabel('Number of iterations')
    plt.show()

def physical_data(Q,req,x_V,y_V,colormap='jet',levels=20):
    """
    Plot all the data required by the user
    :param Q: state vector (3D-array)
    :param req: (list) of (str) defining the required plots
    :param x_V: x coordinates of the cell centers (2D-array)
    :param y_V: y coordinates of the cell centers (2D-array)
    :param colormap: colormap used on the plot (str)
    :param levels: number of level lines required on the plots (int)
    """
    parameters = {'axes.labelsize': 20,
                  'axes.titlesize': 20,
                  'legend.fontsize': 20,
                  'xtick.labelsize': 15,
                  'ytick.labelsize': 15}
    plt.rcParams.update(parameters)
    for x in req:
        plots = x.split(';')
        for p in plots:
            if p == 'rho':
                plt.contourf(x_V, y_V, Q[:, :, 0], levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'rhou':
                plt.contourf(x_V, y_V, Q[:, :, 1], levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'rhov':
                plt.contourf(x_V, y_V, Q[:, :, 2], levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'rhoet':
```

```
                plt.contourf(x_V, y_V, Q[:, :, 3], levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'u':
                plt.contourf(x_V, y_V, extract.u(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'v':
                plt.contourf(x_V, y_V, extract.v(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'et':
                plt.contourf(x_V, y_V, extract.et(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'P':
                # can be converted in hpa or be given relative to the
initial value
                plt.contourf(x_V, y_V, extract.P(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'P0':
                # can be converted in hpa or be given relative to the
initial value
                plt.contourf(x_V, y_V, extract.P0(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'c':
                plt.contourf(x_V, y_V, extract.c(gamma,extract.T(Q)),
levels=levels, cmap=colormap)
                plt.colorbar()
            elif p == 'T':
                plt.contourf(x_V, y_V, extract.T(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'ht':
                plt.contourf(x_V, y_V, extract.ht(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'M':
                plt.contourf(x_V, y_V, extract.M(Q), levels=levels,
cmap=colormap)
                plt.colorbar()
            elif p == 'quiver':
                # this plot does not look nice and could be improved
                plt.quiver(x_V, y_V, extract.u(Q), extract.v(Q))
        plt.xlabel('x')
        plt.ylabel('y')
        plt.title(plots)
        plt.show()
```

Appendix E: save file

```
def log(path,file,nb_iter,dateini,date_final,simulation_time,converged):
    """
    Saves the log of the simulation in the chosen file
    :param path: path to the save folder (str)
    :param file: name of the save file (str)
    :param nb_iter: number of iterations of the simulation (int)
    :param dateini: date of the beginning of the simulation (str)
    :param date_final: date of the end of the simulation (str)
    :param simulation_time: length of the simulation in seconds (float)
    :param converged: flag to indicate if the simulation converged (bool)
    """
    with open(path+'/'+file, 'w') as fout:
        fout.write('Number of iterations: '+str(nb_iter)+'\n')
        fout.write('Beginning of simulation: '+dateini+'\n')
        fout.write('End of simulation: ' + date_final + '\n')
        fout.write('Time of simulation (seconds): ' + str(simulation_time)
+ '\n')
        fout.write('Simulation convergence: ' + str(converged) + '\n')


def residuals(path,file,Cr,Xr,Yr,Er):
    """
    Saves the residuals in the chosen file
    :param path: path to the save folder (str)
    :param file: name of the save file (str)
    :param Cr: continuity residual (list)
    :param Xr: X-momentum residual (list)
    :param Yr: Y-momentum residual (list)
    :param Er: energy residual (list)
    """
    with open(path+'/'+file, "w") as fout:
        fout.write("continuity residual;x momentum residual;y momentum
residual;energy residual"+'\n')
        for i in range(len(Cr)):

fout.write(str(Cr[i])+';'+str(Xr[i])+';'+str(Yr[i])+';'+str(Er[i])+'\n')

def computational_time(path,file,T):
    """
    Saves the simulation time in the chosen file
    :param path: path to the save folder (str)
    :param file: name of the save file (str)
    :param T: list of simulation time of each time steps (list)
    """
    with open(path+'/'+file, "w") as fout:
        for i in range(len(T)):
            fout.write(str(T[i])+'\n')

def states(path,file,X,nx,ny):
    """
    Saves the state vectors and the data required for reconstruction in the
chosen file
    :param path: path to the save folder (str)
    :param file: name of the save file (str)
    :param X: matrix of state vectors (3D-arrays)
    :param nx: dimension of the second axis of the array X (int)
    :param ny: dimension of the first axis of the array X (int)
    """
    rho = X[:,:,0].flatten()
```

```
    rho_u = X[:, :, 1].flatten()
    rho_v = X[:, :, 2].flatten()
    rho_et = X[:, :, 3].flatten()
    with open(path+'/'+file, "w") as fout:
        fout.write('nx:'+str(nx)+' ny:'+str(ny)+'\n')
        fout.write("rho;rho*u;rho*v;rho*et"+'\n')
        for i in range(len(rho)):

fout.write(str(rho[i])+';'+str(rho_u[i])+';'+str(rho_v[i])+';'+str(rho_et[i
])+'\n')
```

Appendix F: step_1_AUSM file (computes only one time step)

```
import numpy as np
import matplotlib.pyplot as plt

import readgrid
import cellproperties

import plotgrid

######################### change boundary conditions with the new functions
made(T and P)

#########################################################################
###################  air thermo data definition  ######################
#########################################################################
from air import *

#########################################################################
#################  initial conditions definition  ######################
#########################################################################
from initial_conditions import *

#########################################################################
###################  physical data extraction  ##########################
#########################################################################
import extract

#########################################################################
##########################  MUSCL scheme  ###############################
#########################################################################
import interpolation

#########################################################################
################  Initialisation and boundaries  ########################
#########################################################################
import set


#########################################################################
##########################  AUSM scheme  ################################
#########################################################################

def M_plus(M):
    """
    Computes the value of M+ as given in the AUSM scheme
    :param M: mach number (2D-array)
    :return: M+ (2D-array)
    """
    return((np.abs(M)<=1)*(M+1)**2/4 + (np.abs(M)>1)*(M+np.abs(M))/2)
    y, x = np.shape(M)
    res = np.zeros((y, x))
    for j in range(y):
        for i in range(x):
            if np.abs(M[j, i]) <= 1:
                res[j, i] = ((M[j, i] + 1) ** 2) / 4
            else:
                res[j, i] = (M[j, i] + np.abs(M[j, i])) / 2
    return (res)

def M_minus(M):
```

```
    """
    Computes the value of M- as given in the AUSM scheme
    :param M: mach number (2D-array)
    :return: M- (2D-array)
    """
    return ((np.abs(M) <= 1) * (-(M - 1)**2) / 4 + (np.abs(M) > 1) * (M -
np.abs(M)) / 2)
    y, x = np.shape(M)
    res = np.zeros((y, x))
    for j in range(y):
        for i in range(x):
            if np.abs(M[j, i]) <= 1:
                res[j, i] = (-(M[j,i]-1)**2)/4
            else:
                res[j, i] = (M[j, i]-np.abs(M[j,i])) / 2
    return (res)

def P_plus(P,M):
    """
    Computes the value of P+ at the first order as given in the AUSM scheme
    :param P: pressure (2D-array)
    :return: P+ at first order (2D-array)
    """
    return((np.abs(M)>1)*(P/2*(1+np.sign(M))) + (np.abs(M)<=1)*(P/2*(1+M)))
    y,x = np.shape(P)
    res = np.zeros((y,x))
    for j in range(y):
        for i in range(x):
            if np.abs(M[j,i]) <= 1:
                res[j,i] = P[j,i]/2*(1+M[j,i])
            else:
                res[j,i] = P[j,i]/2*(M[j,i]+np.abs(M[j,i]))/M[j,i]
    return(res)

def P_minus(P,M):
    """
    Computes the value of P- at the first order as given in the AUSM scheme
    :param P: pressure (2D-array)
    :return: P- at first order (2D-array)
    """
    return ((np.abs(M) > 1) * (P / 2 * (1 - np.sign(M))) + (np.abs(M) <= 1)
* (P / 2 * (1 - M)))
    y, x = np.shape(P)
    res = np.zeros((y, x))
    for j in range(y):
        for i in range(x):
            if np.abs(M[j, i]) <= 1:
                res[j, i] = -P[j, i] / 2 * (1 - M[j, i])
            else:
                res[j, i] = P[j, i] / 2 * (M[j, i] - np.abs(M[j, i])) /
M[j, i]
    return (res)

def phi(X):
    """
    Computes the vector called phi in the AUSM schem divided by Mach 1/2
    :param X: state vector (3D-array)
    :return: phi/M_half (3D-array)
    """
    a = extract.c(gamma,extract.T(X))
    res = np.zeros(np.shape(X))
```

```
    rho = X[:,:,0]
    rho_u = X [:,:, 1]
    rho_v = X[:,:,2]
    ht = extract.ht(X)
    res[:,:,0] = rho*a
    res[:,:,1] = rho_u*a
    res[:,:,2] = rho_v*a
    res[:,:,3] = rho*ht*a
    return(res)

def one_step(X,deltat,V,Sb,Sl,x_V,y_V,nx_xi,ny_xi,nx_eta,ny_eta,plot,iter):
    """
    Updates the array of state vectors after one time step of length deltat
    :param X: array of state vectors (3D-arrays)
    :param deltat: time step (float)
    :param V: array of cells volume (2D-array)
    :param Sb: array of surface vectors of surfaces parallel to xi (3D-
array)
    :param Sl: array of surface vectors of surfaces parallel to eta (3D-
array)
    :param x_V: x coordinates of cells center (2D-array)
    :param y_V: y coordinates of cells center (2D-array)
    :param nx_xi,ny_xi,nx_eta,ny_eta: direction vector of the surfaces (2D-
array)
    :param plot: flag set to 1 if we want a plot of the grid after the
computed time step (bool)
    :param iter: the iteration number of the plot made (int)
    :return: updated array of state vectors X (3D-arrays)
    """

    # extrapolate Q on cell edges with MUSCL scheme #
    Q_l_x_plus_half,Q_r_x_plus_half,Q_l_y_plus_half,Q_r_y_plus_half =
interpolation.MUSCL(X)  # we can use the function MUSCL_V1 to have faster
computations

    # Use extrapolated Q to compute E and F as given in the AUSM scheme #
        # E_xi
            # compute Mxi1/2
    u_l = extract.u(Q_l_x_plus_half)
    v_l = extract.v(Q_l_x_plus_half)
    u_xi_l = nx_xi*u_l + ny_xi*v_l
    a_l_x = extract.c(gamma,extract.T(Q_l_x_plus_half))
    M_xi_l_plus = M_plus(u_xi_l/a_l_x)

    u_r = extract.u(Q_r_x_plus_half)
    v_r = extract.v(Q_r_x_plus_half)
    u_xi_r = nx_xi * u_r + ny_xi * v_r
    a_r_x = extract.c(gamma, extract.T(Q_r_x_plus_half))
    M_xi_r_minus = M_minus(u_xi_r / a_r_x)

    M_xi_half = M_xi_l_plus + M_xi_r_minus

    #advection_E = ((np.abs(M_xi_half) >=
0)[:,:,np.newaxis]*phi(Q_l_x_plus_half) + (np.abs(M_xi_half) <
0)[:,:,np.newaxis]*phi(Q_r_x_plus_half))  ########multiply with M_half
    advection_E =
M_xi_half[:,:,np.newaxis]*0.5*(phi(Q_l_x_plus_half)+phi(Q_r_x_plus_half))-
0.5*np.abs(M_xi_half)[:,:,np.newaxis]*(phi(Q_r_x_plus_half)-
phi(Q_l_x_plus_half))

    # compute the pressure part
```

```
    P_l = extract.P(Q_l_x_plus_half)
    P_l_plus = P_plus(P_l,u_xi_l/a_l_x)
    #P_l_plus = P_plus(P_l, M_xi_half)
    P_r = extract.P(Q_r_x_plus_half)
    P_r_minus = P_minus(P_r,u_xi_r / a_r_x)
    #P_r_minus = P_minus(P_r, M_xi_half)

    pressure = np.zeros(np.shape(Q_l_x_plus_half))
    pressure[:,:,1] = nx_xi*(P_l_plus+P_r_minus)  # as here the computation
averages at cell face, we use extrapolated direction vector at cell face
    pressure[:,:,2] = ny_xi*(P_l_plus+P_r_minus)  # as here the computation
averages at cell face, we use extrapolated direction vector at cell face

    E_xi = advection_E+pressure

        # F_eta
            # compute Meta1/2
    u_l = extract.u(Q_l_y_plus_half)
    v_l = extract.v(Q_l_y_plus_half)
    v_eta_l = nx_eta * u_l + ny_eta * v_l
    a_l_y = extract.c(gamma, extract.T(Q_l_y_plus_half))
    M_eta_l_plus = M_plus(v_eta_l / a_l_y)

    u_r = extract.u(Q_r_y_plus_half)
    v_r = extract.v(Q_r_y_plus_half)
    v_eta_r = nx_eta * u_r + ny_eta * v_r
    a_r_y = extract.c(gamma, extract.T(Q_r_y_plus_half))
    M_eta_r_minus = M_minus(v_eta_r / a_r_y)

    M_eta_half = M_eta_l_plus + M_eta_r_minus

    #advection_F = ((np.abs(M_eta_half) >= 0)[:, :, np.newaxis] *
phi(Q_l_y_plus_half) + (np.abs(M_eta_half) < 0)[:, :,np.newaxis] *
phi(Q_r_y_plus_half))  ##### when we forget to multiply by M_half, it works
but when we multiply it doesn't work
    advection_F =
0.5*M_eta_half[:,:,np.newaxis]*(phi(Q_l_y_plus_half)+phi(Q_r_y_plus_half))-
0.5*np.abs(M_eta_half)[:,:,np.newaxis]*(phi(Q_r_y_plus_half)-
phi(Q_l_y_plus_half))

    # compute the pressure part
    P_l = extract.P(Q_l_y_plus_half)
    P_l_plus = P_plus(P_l, v_eta_l / a_l_y)  ############## vérifier le
mach number à utiliser ici
    #P_l_plus = P_plus(P_l, M_eta_half)
    P_r = extract.P(Q_r_y_plus_half)
    P_r_minus = P_minus(P_r, v_eta_r / a_r_y)  ############## vérifier le
mach number à utiliser ici
    #P_r_minus = P_minus(P_r, M_eta_half)

    pressure = np.zeros(np.shape(Q_l_y_plus_half))
    pressure[:, :, 1] = nx_eta * (P_l_plus + P_r_minus)  # as here the
computation averages at cell face, we use extrapolated direction vector at
cell face
    pressure[:, :, 2] = ny_eta * (P_l_plus + P_r_minus)  # as here the
computation averages at cell face, we use extrapolated direction vector at
cell face

    F_eta = advection_F + pressure # the flux for u or v is certainly too
big to have any convergence
```

```
    # computing the step of the iteration with a Roe scheme #
    i_variation = E_xi[:,1:,:]*np.linalg.norm(Sl[1:-1,2:-
1,:],axis=2)[:,:,np.newaxis] - E_xi[:,:-1,:]*np.linalg.norm(Sl[1:-1,1:-
2,:],axis=2)[:,:,np.newaxis]
    j_variation = F_eta[1:,:,:]*np.linalg.norm(Sb[2:-1,1:-
1,:],axis=2)[:,:,np.newaxis] - F_eta[:-1,:,:]*np.linalg.norm(Sb[1:-2,1:-
1,:],axis=2)[:,:,np.newaxis]
    step = deltat*(i_variation+j_variation)

    # apply the step #
    X[1:-1, 1:-1, :] = X[1:-1, 1:-1, :] - step/V[1:-1,1:-1,np.newaxis]

    # reset the boundary conditions #
    X = set.bc(X, nx_eta,ny_eta)

    # if required, plot intermediate states #
    if plot:
        plt.contourf(x_V, y_V, X[:, :, 0], levels=20, cmap='jet')
        plt.colorbar()
        plt.xlabel('x')
        plt.ylabel('y')
        plt.title('Density')
        plt.quiver(x_V, y_V, extract.u(X), extract.v(X))
        #plt.savefig('C:/Users/bapti/OneDrive/fichiers ancien
pc/OneDrive/Documents/prepa/GeorgiaTech/sur place/spring
2023/CFD/Project/data/output/step_1_AUSM/gif_images/'+str(iter),dpi=500)
        #plt.clf()
        plt.show()

    return(X)
```

Appendix G: step_1 file (full computational loop)

```
import numpy as np
import time
import  matplotlib.pyplot as plt
import cellproperties
import extract
import plot
import readgrid
import save
import set
import step_1_AUSM

def solve(nx,ny,V,Sb,Sl,x_V,y_V):
    """
    Computational loop used to find Q with a given scheme on the chosen
grid
    It returns all parameters used for data saving
    :param nx, ny: grid size (int)
    :param V: list of cells volume (2D-array)
    :param Sb: list of bottom surface vectors (3D-array)
    :param Sl: list of left surface vectors (3D-array)
    :return:
    """
    # initialize simulation parameters #
    deltat = 0.00001 # for unstretched grids,0.00001 is fine. for stretched
mesh, 0.000001 is fine
    nb_iter = 0
    plot_iter = 0
    plot = 0
    nx_xi, ny_xi = extract.direction_vectors_i(Sb, Sl)
    nx_eta, ny_eta = extract.direction_vectors_j(Sb, Sl)
    # initialize data to be saved #
    Cr = []
    Xr = []
    Yr = []
    Er = []
    T = [0]
    tini = time.time()
    dateini = time.ctime()
    converged = False

    # Initialise computational space #
    Q = set.initial_conditions(nx,ny,Sb)
    Q = set.bc(Q, nx_eta,ny_eta)
    Qini = np.copy(Q)
    Q0 = np.copy(Q)
    Q = step_1_AUSM.one_step(Q, deltat,V,Sb,Sl,x_V,
y_V,nx_xi,ny_xi,nx_eta,ny_eta,plot,nb_iter)
    T.append(T[-1]+deltat)
    nb_iter += 1
    plot_iter += 1
    cr,xr,yr,er = extract.residuals_L2(Q,Q0,Qini)
    Cr.append(cr)
    Xr.append(xr)
    Yr.append(yr)
    Er.append(er)
    print(cr)

    # solving loop #
    while cr > 1e-16:  # for the moment the residual are not relative to
```

```
initial conditions so in the end we should look at max(cr,xr,yr,er) but for
the moment as rhoini~1, we use this one to set convergence
        # update Q #
        Q0 = np.copy(Q)
        Q = step_1_AUSM.one_step(Q, deltat,V,Sb,Sl,x_V,
y_V,nx_xi,ny_xi,nx_eta,ny_eta,plot,nb_iter)

        # compute residuals #
        cr, xr, yr, er = extract.residuals_L2(Q, Q0,Qini)
        Cr.append(cr)
        Xr.append(xr)
        Yr.append(yr)
        Er.append(er)
        print(cr)

        # update simulation data #
        nb_iter += 1
        plot_iter += 1
        T.append(T[-1]+deltat)

        # set plot flag #
        if plot_iter > 10000:
            plot = 1
            plot_iter = 0
        else:
            plot = 0
    print(T[-1])
    converged = True
    simulation_time = time.time() - tini
    date_final = time.ctime()

return(Q,T,Cr,Xr,Yr,Er,nb_iter,dateini,date_final,simulation_time,converged
)

if __name__ == '__main__':
    # reading the grid #
    nx, ny, x, y = readgrid.readGrid('../../grids',
'g65x49u_augmented.dat')
    Volumes, x_V, y_V, bottom_Surfaces, x_Sb, y_Sb, left_Surfaces, x_Sl,
y_Sl = cellproperties.cellsCharacteristics('../../grids',
'g65x49u_augmented.dat')
    # solve the grid #
    Q,T,Cr,Xr,Yr,Er,nb_iter,dateini,date_final,simulation_time,converged =
solve(nx,ny,Volumes,bottom_Surfaces,left_Surfaces,x_V, y_V)
    # save data #

save.log('../../data/output/step_1_AUSM','log',nb_iter,dateini,date_final,s
imulation_time,converged)
    save.residuals('../../data/output/step_1_AUSM','residuals',Cr,Xr,Yr,Er)
    save.computational_time('../../data/output/step_1_AUSM','stats',T)
    save.states('../../data/output/step_1_AUSM','states',Q,nx,ny)
    # plot data #
    plot.residuals(Cr,Xr,Yr,Er)
    plot.physical_data(Q,['rho','P','P0','T','M'],x_V, y_V,levels=100)
    parameters = {'axes.labelsize': 20,
                  'axes.titlesize': 20,
                  'legend.fontsize': 20,
                  'xtick.labelsize': 15,
                  'ytick.labelsize': 15}
    plt.rcParams.update(parameters)
    plt.plot(x_Sb[1,:],extract.P(Q)[1,:])
```

```
    plt.xlabel('x')
    plt.ylabel('P')
    plt.show()
    print(Q[7,32,0],Q[1,0,0],Q[7,32,0]/Q[1,0,0])  # rho

print(extract.P(Q)[7,32],extract.P(Q)[1,0],extract.P(Q)[7,32]/extract.P(Q)[
1,0])  # P

print(extract.T(Q)[7,32],extract.T(Q)[1,0],extract.T(Q)[7,32]/extract.T(Q)[
1,0])  # T

print(extract.P0(Q)[7,32],extract.P0(Q)[1,0],extract.P0(Q)[7,32]/extract.P0
(Q)[1,0])  # P0
    print(extract.M(Q)[7,32])  # M
```

Appendix H: initial and physical conditions

```
from air import *
# initial conditions
Tini = 300.0
Pini = 101325.0
Mini = 2.0
rhoini = Pini/r/Tini

# constant thermodynamic data of air
R = 8.31446261815324
MW = 28.96e-3
r = R/MW
gamma = 1.400  # can use NIST data to compute
Cp = 1005  # J/kg/K , can use NIST data to compute
```

Appendix I: Computer architecture and language used

This code has been developed on personal computer operating on Windows system. Machine specification is summarized as shown below:

Cpu name : 12th Gen Intel® core™ i5-1235U 1.30GHz

cpu cores : 10

Memory : 8 GB

Language used: Python